University of Southern Denmark

Bachelor Project

IMADA

Analysis of Real-World Developer Mistakes in configurable software systems

Student:
Salma Khair

Supervisor:
Dr. rer. nat. Sandra Greiner
Associate Professor

December 19, 2025

Contents

1 Introduction **1**

2 Background and State-Of-The-Art **5**
2.1 Variable Software Development 5
2.1.1 Principles of Software Product Lines 6
2.1.2 Classification of Edits to Configurable Software 6
2.2 Principles of Natural Language Processing 6
2.2.1 Basics 6
2.2.2 Topic Modeling 6

3 Implementation **7**
3.1 Preliminaries 7
3.1.1 DiffDetective 7
3.2 Architecture 7
3.3 Evaluation 7
3.3.1 Setup 7
3.3.2 Results 7
3.3.3 Threats to Validity 7

4 Discussion **8**

5 Conclusion **9**

Chapter 1 Introduction

Modern software systems are rarely developed as a single, fixed program. Instead, many systems are configurable, meaning that the same code base can be compiled into different software variants depending on hardware, platform, or selected features. A common way to implement such configurability is through conditional code guarded by presence conditions, for example using preprocessor directives such as `#if` and `#ifdef`. Presence conditions determine in which configurations a particular code fragment is included and therefore directly influences the behavior of the resulting software variants.

This bachelor thesis is situated in the area of software engineering, with a focus on configurable software systems and variability analysis. It studies how changes to conditional code, such as presence conditions e.g. `#ifdef`, affect software systems over time by analysing large software repositories. In standard version control systems, changes are represented as textual diffs that show added or removed lines of code. While this representation is sufficient for many development tasks, it is often insufficient for configurable systems. In such systems, even a small textual change can have a large semantic impact if it modifies a presence condition.

For example, a minor edit to a conditional expression may silently expand or restrict the set of configurations in which a code fragment is active. These effects are not visible in standard diffs and are often undocumented in commit messages. As a result, developers and tools may overlook unintended changes to variability, which can lead to configuration of specific faults that are difficult to detect. The problem addressed in this thesis is therefore the limited ability of traditional diffs to explain the impact of changes to presence conditions. In particular, the thesis investigates how developer mistakes related to variability can be identified by analyzing commits at the level of variability semantics rather than raw text differences.

To address this problem, this thesis builds on the DiffDetective tool to explicitly identify changes in variability. DiffDetective analyzes software evolution by mapping source code and annotations to structured representations and by classifying edits according to their effect on variability. Instead of treating all changes as simple additions or deletions, DiffDetective distinguishes different kinds of variability edits, such as refactoring and reconfiguration. Based on this foundation, this thesis contributes a custom analysis tool called `MyAnalysis`. The purpose of `MyAnalysis` is to identify and store variability edits that are relevant for understanding developer mistakes. The recorded results are later analyzed using a topic-based approach, which groups refactoring and reconfiguration edits and collecting their diff messages. This makes it easier to study variability related diffs in more detail and to reason about recurring patterns in how such changes occur. The analysis focuses mainly on two classes of edits: refactoring and reconfiguration. Refactoring describes changes where the structure of a presence condition is modified while its meaning remains the same. Reconfiguration describes changes where the artifact remains unchanged, but the set of configurations in which it is included changes in a non-trivial way.

In addition, `MyAnalysis` extracts commit-level information such as commit identifiers and commit messages. A simple keyword-based heuristic is used to flag commits that are likely related to bug fixes. This allows the analysis to connect variability edits with developer intent as expressed in

commit messages.

The solution is evaluated by applying `MyAnalysis` to a real-world configurable software system. By analysing how often variability related diffs occur in practice and by examining which types of diffs are most common and studying the distribution of different variability edit types across large software repositories, the evaluation makes it possible to assess the overall extent of the problem. This approach allows us to determine whether changes to presence conditions occur frequently and whether they represent a more substantial issue than initially expected. The results show that the analysis is able to distinguish between different types of variability changes and provides a more transparent view of how presence conditions evolve over time. By going beyond traditional textual diffs, the approach makes variability changes visible that would otherwise be difficult to recognise in standard version control output.

In the following chapters, this thesis is structured as follows. The next chapter provides background on configurable software and presence Conditions and explains the motivation for using variability aware differencing. The subsequent chapter describes the design and implementation of `MyAnalysis`. After that, the evaluation chapter presents the study and discusses the results. Finally, the thesis concludes with a summary of the main findings and an outline of possible directions for future work.

trying with novel page

and another one

Chapter 2 Background and State-Of-The-Art

2.1 Variable Software Development

Moderns software systems are rarely developed as a single, fixed program with one well defined behavior. Instead, many systems are designed as configurable systems in order to support different platforms, environments and user or customer requirements within a single code base. Rather than maintaining multiple separate implementation for each possible variant, developers introduce variability into the software itself, allowing different configurations to be derived from the same source code.

This approach is widely used in the real world software development. Operating systems, system libraries, embedded software and server application often need to adapt to different hardware architecture, operating system, performance requirements or optional features. For example, an operating system kernel may include different code depending on whether it is built for a specific processor architecture, while a network server may enable or disable certain features based on configuration options. Supporting such variability within a single code base makes it possible to reuse large parts of the implementation and reduces the need to maintain separate forks of the same system.

At the same time, configurability introduces additional complexity. Reasoning about a single program, developers must consider a potentially large number of software variants. Each configuration represents a specific combination of features and may include or exclude different parts of the code. In large systems, the number of valid configurations can easily grow into the thousands or even millions, making it infeasible to test or inspect each variant individually. As a result, developers typically reason about variability symbolically rather than concretely, relying on conditions that describe when certain code should be active.

In practice, variability is often implemented using conditional code. In languages such as C and C++, this is commonly done through preprocessor directives such as `#if`, `#ifdef`, and `#ifndef`. These directives are used to guard code fragments with logical conditions, also referred to as presence conditions. A presence condition describes when a specific piece of code becomes part of a software variant. In practice, such conditions are often expressed as logical expressions and can be combined to represent more complex configuration rules.

While conditional compilation is a powerful mechanism, it also makes the code harder to understand and maintain. Small changes to a presence condition can have far reaching consequences, as they may affect many configurations at once. Developers working on a specific change often focus on the configuration they are currently interested in and may not be fully aware of how the same change impacts other variants of the system. This makes configurable software particularly open to subtle and unintended errors.

The complexity introduced by variability becomes especially apparent during software evolution. As systems evolve over time, developers continuously modify, extend and refactor code. In con-

figurable systems, such edits not only change the implementation itself but may also alter the structure of variability, that is, the relationship between features, conditions, and code fragments. For example, a seemingly minor modification to a conditional expression can unintentionally expand or restrict the set of configurations in which a piece of code is active.

These effects are often not visible in standard version control diffs, which typically represent changes as simple additions or deletions of lines of code. As a result, developers may overlook important changes to variability, leading to configuration-specific faults that are difficult to detect and diagnose. Understanding how changes to presence conditions affect the overall variability of a system is therefore crucial for maintaining the correctness and reliability of configurable software.

2.1.1 Principles of Software Product Lines

2.1.2 Classification of Edits to Configurable Software

Edits to variable software can fully classified in 8 categories [1].

2.2 Principles of Natural Language Processing

2.2.1 Basics

2.2.2 Topic Modeling

Chapter 3 Implementation

3.1 Preliminaries

3.1.1 DiffDetective

3.2 Architecture

3.3 Evaluation

3.3.1 Setup

3.3.2 Results

3.3.3 Threats to Validity

[2]

Chapter 4 Discussion

Chapter 5 Conclusion

List of Figures

List of Tables

Bibliography

[1] Paul Maximilian Bittner et al. "Classifying edits to variability in source code". In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2022. Singapore, Singapore: ACM, 2022, pp. 196–208. DOI: 10.1145/3540250.3549108.

[2] Claes Wohlin et al. *Experimentation in Software Engineering, Second Edition*. Springer, 2024. DOI: 10.1007/978-3-662-69306-3.